

Week 5: Running the Flutter Apps in VS Code

Dr M AOUE

Guide to the week 5 programs

About this guide. It explains how to run the week 5 Flutter apps in **Visual Studio Code**, and describes what each app should show on the screen and how to try it. This week the apps react to you: you type in text fields and press buttons. Each file starts with a comment that says what it teaches and what result to expect: run it, use it, then compare with that comment.

1 What you need

- **Flutter** installed, and **Visual Studio Code** with the **Flutter** extension. If something is missing, follow the install guide 001_install_vscode. Check with `flutter doctor` in a terminal.
- **A device to run on.** **Chrome** or **Windows (desktop)** is enough: you type with your computer keyboard. An Android emulator or a phone is closer to reality, because it shows the on-screen keyboard (for example the numeric keyboard of the sum app).

2 Get the week 5 code and open it

If you already copied the repository, update it: open a terminal in the repository folder and type

```
git pull
```

Otherwise download it with `git clone https://github.com/maoude/mobile_pro_2026.git`. In VS Code choose **File** → **Open Folder...** and open the project folder `code/week05/flutter_examples` (the second project of the week, `code/week05/examples`, is described in Section 6). Open the folder of the project itself, not a parent folder: this is how VS Code recognises a Flutter project.

3 Prepare the project (once)

The folder contains only the Dart code. Open a terminal in it (**Terminal** → **New Terminal**) and type:

```
flutter pub get  
flutter create .
```

`flutter pub get` downloads the packages. `flutter create .` adds the platform folders (`android`, `web`, `windows...`) that are needed to run the app. It does not change the existing files. You only do this once.

4 Choose a device

At the bottom right of the VS Code window, click the device name (it may say **No Device**) and choose **Chrome, Windows (desktop)**, or an Android emulator if you created one in Android Studio's Device Manager.

5 Run an app

Every file in the folder `lib` is a complete app with its own `main`:

1. Open the file, for example `lib/03_you_typed.dart`.
2. Click **Run** in the small **Run | Debug** line above `void main()`. The first start takes a little while.
3. The app opens in a window (or in Chrome). The file `lib/main.dart` starts the sum app, the last app of the lecture (see Section 6).

The sum app is in a *folder*, because it is written in three files (`main.dart`, `home.dart`, `my_text_field.dart`). To run it, open `lib/05_sum_app/main.dart` and click **Run** above its `main`. From a terminal, run any file with the `-t` option: `flutter run -t lib/03_you_typed.dart`, or `flutter run -t lib/05_sum_app/main.dart`.

Change the code and see the result at once

While the app runs, change the code and save the file (`Ctrl+S`): Flutter updates the running app in a second. This is called **hot reload**. This week, notice that *hot reload keeps the state*: the text you typed and the counters keep their values, because the state object is not destroyed. `Ctrl+Shift+F5` does a **hot restart**, which starts the app again from the beginning, with the initial values. `Shift+F5` stops it.

Run the automated checks

The project also contains tests, in the folder `test`. Each test starts one app, types in the text fields and presses the buttons like a user, and checks what appears on the screen. Type `flutter test`: it should end with **All tests passed!**

6 The apps: what you should see and try

Run them in order.

File	What you see and what to try
<code>00_stateful_counter.dart</code>	"Count: 0" and a button Add 1 . Press it: the number goes up by 1 each time. This is the smallest stateful widget.
<code>01_setstate_needed.dart</code>	"Value: 0" and two buttons. Press without setState three times: nothing changes. Press with setState once: the value jumps to 4.
<code>02_textfield_events.dart</code>	A text field with a frame, a label, an icon and a hint, and two lines "Changed:" and "Submitted:". Type: the first line follows every letter. Press Enter: the second line shows the text.

03_you_typed.dart	“You typed:” and a text field. Type “Hello students”: the text changes after each letter. This is the first application of the lecture.
04_custom_textfield.dart	The same screen as the previous one, but the field is now your own widget <code>MyTextField</code> , which calls a callback function.
05_sum_app/main.dart	“Sum:”, two fields “Enter X” and “Enter Y”, and a button SUM . Type 2.5 and 4, press the button: “Sum: 6.5”. Leave a field empty, or type letters, and press the button: “Sum: Please fill all fields”. It is also the app that <code>main.dart</code> starts.

Going further: eight more apps

The buttons app answers the exercise of the slides; the others are not in the slides. Three of them (10 to 12) build one app, the **Stopwatch**, step by step: run them in order.

File	What you see and what to try
06_text_controller.dart	A field that already contains “Flutter”, the line “Characters: 7” and buttons UPPERCASE and CLEAR . Type more letters: the count follows. UPPERCASE changes the text, CLEAR empties the field.
07_button_styles.dart	Five buttons: a default one, a violet “pill”, a green button with rounded corners, a red circle with a heart, and a grey DISABLED button. The first line shows the last button pressed. Turn on the switch: the last button becomes active.
08_safe_number_input.dart	Two fields and a grey SUM button. Type “abc” in X: a red message “Not a number” appears and the button stays grey. Type 2.5 in X and -4 in Y: the button becomes active; press it: “Sum: -1.5”.
09_lifecycle_log.dart	A card “Child: A”, two buttons, and a numbered log. Press Change label , Hide child and Show child in turn, and read the order of the life cycle methods in the log. Press <code>r</code> in the terminal (hot reload): reassemble appears.
10_stopwatch_timer.dart	“0 seconds” in the middle of the screen. It counts up by itself, once a second, with no button.
11_stopwatch_buttons.dart	The same counter, with a green Start and a red Stop . Press Start : it counts; Stop is now active and Start is grey. Press Stop , then Start again: it restarts from 0.
12_stopwatch_laps.dart	A blue panel (“Lap 1”, the time, and Start / Lap / Stop) above an empty list. Start it, then press Lap a few times: each lap is added to the scrolling list below, and the panel moves to the next lap.
13_scrolling.dart	A drop-down “Physics” and a list of 1000 rows. Drag the list to scroll it; choose Bouncing or Never and compare how the list reacts at its top.

Part 2: gestures and state

These ten apps go with the document *Week 5, Part 2: Gestures and State*. They are in a *second* project, `code/week05/examples`: open that folder in VS Code, and prepare it as in Section 3. **Type flutter pub get there**: the last example uses the `provider` package, which must be downloaded (you need an Internet connection the first time).

File	What you see and what to try
<code>00_pointer_events.dart</code>	A square and the line “Last event”. Touch (or click) the square, move, release: the last event and four counters (down, move, up, cancel) change.
<code>01_tap_gestures.dart</code>	A box and six counters. Tap once: after a short pause (0.3 s) three counters go to 1. Tap twice quickly: double tap. Press and hold: long press. Press and slide out of the box: tap cancelled.
<code>02_drag_gestures.dart</code>	A red square on a horizontal track, a green one on a vertical track. Drag each along its track; it stops at the ends.
<code>03_pan_ball.dart</code>	An orange ball in a grey square. Drag it in any direction; RESET puts it back in the middle.
<code>04_pinch_scale.dart</code>	A blue square. Pinch with two fingers to zoom (0.5 to 3 times) and turn them to rotate. On a computer use an emulator (hold Ctrl to show two fingers).
<code>05_gesture_arena.dart</code>	Two yellow squares with an orange square inside. On the left, tap the orange square: only the child counter changes. On the right, both counters change.
<code>06_ephemeral_state.dart</code>	A button that changes its name (Rami / Lina), and three hearts: each heart has its own state.
<code>07_lifting_state_up.dart</code>	A cart count in three places. Press Add : all three change at once.
<code>08_change_notifier.dart</code>	A counter with + and -. The line “Page built” stays at 1 while “Text built” grows: only the text is rebuilt.
<code>09_provider_cart.dart</code>	A catalog: add products with the cart buttons and watch the badge. Open the cart (icon): the items, the total, remove buttons. The moon icon switches both pages to the dark theme.

Part 3: forms and validation

These three apps go with the document *Week 5, Part 3: Forms and Validation*. They are in the *same* second project, `code/week05/examples`, so no extra setup is needed once Part 2 is prepared.

File	What you see and what to try
<code>10_dropdown_button.dart</code>	The text “You live in: Calcutta” and a dropdown. Tap it, pick another city: the text updates.

11_form_validation.dart	Two fields, a city dropdown, and SUBMIT / RESET . Press SUBMIT with empty fields: two red messages appear. Fill them in and press it again: a result line appears. Press RESET : everything clears.
12_more_validators.dart	Email, Password and Confirm password fields, and a CHECK button. Type in a field: its own error appears live, before you press anything. Make all three valid, then press CHECK .
13_input_formatters.dart	A field with the hint “6-digit code”. Try to type letters: nothing appears. Type digits: they appear, and stop being accepted after 6.
14_controller_listener.dart	A field and the line “Mirrored by addListener”. Type: it follows. Press the button below: the field jumps to “Hello” by itself, and the mirrored line follows that too.
15_form_of_builder.dart	A field “Your name” and a button VALIDATE (Form.of) . Press it empty: a red message and “valid: false”. Fill it in, press again: “valid: true”.
16_custom_formfield.dart	A boxed 6-digit field and SUBMIT . Press it empty, then with “123”: two different red messages. Type all 6 digits and press SUBMIT : “Code accepted: ...” appears and the field clears.

Try it yourself

Change the code and watch the result after each save:

- in 01_setstate_needed.dart, put the second button’s line inside a **setState**: what changes?
- in 02_textfield_events.dart, add **obscureText: true** and **maxLength: 8** to the **TextField**;
- in 04_custom_textfield.dart, change the font size and the border of **MyTextField**, and add a second field with another hint;
- in 05_sum_app, add a third field **Z**, then a button that multiplies;
- in 07_button_styles.dart, change the colors and the **shape** of a button; make the **DEFAULT** button round;
- in 01_tap_gestures.dart of the second project, remove **onDoubleTap** and see that the tap is no longer delayed;
- in 08_change_notifier.dart, press **+** and look at the two “built” counters;
- in 09_lifecycle_log.dart, press **Hide child** then **Show child**: read **deactivate**, **dispose**, then a fresh **initState** in the log;
- in 12_stopwatch_laps.dart, change 100 (the tick) to 1000 milliseconds and watch the laps slow down;
- in 13_scrolling.dart, remove the **Expanded** around the **ListView** and read the error in the Debug Console;
- in 10_dropdown_button.dart, add a sixth city to **_cities**;
- in 11_form_validation.dart, press **SUBMIT** with only the name filled in: only the age field shows an error;
- in 12_more_validators.dart, type a short password, then a longer one, and watch the error appear and disappear as you type;
- in 13_input_formatters.dart, try to paste (or type) more than 6 digits;

- in `14_controller_listener.dart`, press the button several times in a row and watch the mirrored line every time;
- in `16_custom_formfield.dart`, type a 6-digit code, press **SUBMIT**, and watch the field clear itself;
- type text in one of the apps, change some code, and save: the text you typed stays (hot reload). Then press `Ctrl+Shift+F5`: it is gone (hot restart).

7 Quick troubleshooting

Problem	What to do
No Run Debug line above <code>main</code>	Install the Flutter extension, and open the project folder <code>flutter_examples</code> itself.
The device list at the bottom right says No Device	Click it and choose Chrome . Check that <code>flutter doctor</code> shows Chrome.
An error says that no platform is configured	Run <code>flutter create .</code> once in the project folder (Section 3).
The sum app in <code>05_sum_app</code> does not start	Run <code>lib/05_sum_app/main.dart</code> (not <code>home.dart</code>): only <code>main.dart</code> has a <code>main</code> function.
Nothing changes when you press a button in <code>01_setstate_needed.dart</code>	That is the point of the example: the second button does not call <code>setState</code> .
The laps list of <code>12_stopwatch_laps.dart</code> is empty	Lap is disabled until you press Start .
The list of <code>13_scrolling.dart</code> does not scroll	Check the Physics drop-down: Never disables the finger scroll on purpose.
An error says <i>Target of URI doesn't exist: package:provider/provider.dart</i>	In the folder <code>code/week05/examples</code> , type <code>flutter pub get</code> to download the <code>provider</code> package.
The pinch of <code>04_pinch_scale.dart</code> does nothing	A mouse is one finger. Use an Android emulator and hold <code>Ctrl</code> to show two fingers, or a phone.
SUBMIT in <code>11_form_validation.dart</code> does nothing visible	Look under the two fields: a red message means the form is not valid yet. Fill them in and press it again.
Errors in <code>12_more_validators.dart</code> appear before you press any button	That is the point of the example: <code>autovalidateMode: AutovalidateMode.onUserInteraction</code> checks a field as soon as you touch it.
Letters do not appear in <code>13_input_formatters.dart</code> or the code field of <code>16_custom_formfield.dart</code>	That is the point of the example: an <code>inputFormatter</code> keeps only digits.
<i>Form.of() was called with a context that does not contain a Form widget</i>	The <code>context</code> used is above the <code>Form</code> . Wrap the caller in a <code>Builder</code> , as example <code>15_form_of_builder.dart</code> does.

The numeric keyboard does not appear	On Chrome or Windows you use your computer keyboard. The numeric keyboard appears on an Android emulator or a phone.
Pressing Enter does nothing	Only <code>02_textfield_events.dart</code> reacts to Enter (<code>onSubmitted</code>).
A red error screen after a change	Read the message at the top: it usually names the file and the line. Undo the last change, or press <code>Ctrl+Shift+F5</code> to restart.
Waiting for another flutter command to release the startup lock	Another Flutter command is running, for example in another window. Wait for it, or close it.

This document is maintained by Dr. Mohamad AOUDÉ
Last updated: 2026-09-22